

BT5151 Advanced Analytics and Machine Learning

RouteIQ: A Multi-Agent ML System for Intelligent Customer Support Routing



Name	Student ID
Liu Qichen	A0318602L
Nourah Algiffari	A0317916X
Pranay Samineni	A0318642A
Tanya Tooley	A0318087X
Wang Ying	A0318271H
Zhao Qiya	A0331091R

National University of Singapore

Submission Date: April 17, 2026

1 System Architecture

RouteIQ is a multi-agent ML system that delivers three outputs for every incoming support ticket: a **queue routing decision** with a confidence score, **additional customer context** drawn from CRM records and similar historical tickets, and an **LLM-generated draft reply** ready for the assigned agent to review and send. These three outputs are produced by a single LangGraph pipeline consisting of eight agent nodes, each governed by its own `SKILL.md` policy file. All nodes communicate through a shared `AgentState` TypedDict, a single centralised state store that every node reads from and writes to, ensuring the entire pipeline operates on one consistent view.

The pipeline runs in two phases. The **offline training phase** (Nodes 1–4) runs once: it preprocesses the ticket dataset, trains and evaluates two candidate ML models on the same held-out test set, and selects the best one with a written rationale. The **online inference phase** (Nodes 5–8) runs per incoming ticket: it classifies the ticket to a support queue, retrieves the top-3 most similar past tickets and their agent responses from the training corpus (RAG), looks up the customer’s order record from the mock CRM, and passes all of that context to the LLM to generate the final routing recommendation and draft reply.

The full pipeline diagram is shown in Appendix A. Each pipeline stage is defined by a dedicated `SKILL.md` file. The SKILL design decisions are discussed in Section 5.

2 Business Case

Enterprise software companies receive thousands of support tickets daily spanning billing disputes, technical faults, service outages, and account changes. Manual triage introduces two consistent failure modes: misrouting, where a ticket reaches the wrong team and requires at least one transfer, and delay, where tickets queue for review before any assignment is made. Under manual workflows, 30 to 40% of tickets are misrouted on first assignment [3], costing at least \$22 each in additional handling against a baseline cost of \$18 to \$60 per ticket [2]. RouteIQ automates triage using a trained ML pipeline. A rules-based router cannot generalise across 10 semantically overlapping queues in two languages without constant manual maintenance, and a zero-shot LLM classifier adds per-ticket API cost with no reproducible evaluation baseline. A trained TF-IDF classifier evaluated on held-out data provides measurable, auditable routing accuracy before deployment and can be retrained as ticket patterns evolve. RouteIQ classifies each incoming ticket, attaches a confidence score, retrieves similar resolved tickets for context, and delivers a plain-English routing recommendation to the receiving agent. Tickets below a 50% confidence threshold are flagged for human review. Its value lies in eliminating triage delay, scaling to arbitrary ticket volume without additional headcount, and delivering consistent routing around the clock. Three stakeholder groups benefit directly. Support managers gain visibility into routing confidence distribution to identify systematically ambiguous ticket types. Frontline agents receive tickets pre-loaded with customer context and a draft response. End customers reach the correct team faster without being passed between queues.

3 Data and ML Methodology

3.1 Dataset and Label Selection

The dataset used is the Multilingual Customer Support Tickets dataset, sourced from Kaggle [1]. After removing 7 rows with missing `answer` fields, the working dataset contains 28,580 tickets across 16 columns. Full column descriptions and usage decisions are provided in Appendix E. The language distribution is 57% English and 43% German.

The target variable is `queue` (10 classes) rather than `type` (4 classes). The routing use case requires directing a ticket to the correct specialist team, a decision governed by *what the ticket is about*, not *how it is categorised* administratively. The `type` field (Incident, Request, Problem, Change) is passed through agent state for use by `draft-response` to calibrate reply tone.

3.2 Preprocessing

Both English and German tickets were retained. Excluding German would discard 43% of the dataset. The `subject` and `body` fields were concatenated into a single `text` field (missing subjects filled with empty strings). The `answer` column was excluded from features to prevent target leakage, as it is written *after* routing and is instead used as the RAG corpus. Tag columns (`tag_1`–`tag_8`) were excluded for the same reason: they are assigned alongside routing, not submitted by the customer.

Queue labels were integer-encoded using `LabelEncoder`. Data was split into train/validation/test (80/10/10) using stratified sampling to preserve class proportions across all three splits. The dataset has a $20.6\times$ class imbalance ratio (see Appendix D). `class_weight='balanced'` was applied in both models to penalise minority-class errors proportionally.

3.3 TF-IDF Vectorisation

Text features were extracted using `TfidfVectorizer` with the following configuration: `max_features=10000`, `ngram_range=(1,2)`, `sublinear_tf=True`, `min_df=2`, and combined English and German stopwords. German umlauts (e.g. ü, ä, ö) are preserved as distinct tokens. No accent normalisation is applied. The vectoriser was fitted exclusively on the training set and applied (without refitting) to the validation and test sets, preventing any leakage of test vocabulary into the feature space.

3.4 Model Training

Two candidate models were trained on the same split and tuned with `GridSearchCV` using `PredefinedSplit` (validation set held fixed throughout). Both were compared against a dummy classifier baseline that always predicts the majority class. Full configurations and hyperparameter search spaces are shown in Appendix F.

4 Model Evaluation

4.1 Summary Metrics

Both trained models substantially outperform the baseline across all metrics (see Appendix B). Random Forest is selected as the production model on the basis of Macro F1 (0.527 vs 0.511),

which treats all 10 queue classes equally regardless of their size. This is the appropriate criterion for a routing system where misclassifying a minority queue is just as operationally costly as misclassifying the majority. Full per-class Precision, Recall, and F1 are reported in Appendix B. Confusion matrix heatmaps and the per-class F1 bar chart are in Appendix C.

4.2 Business Interpretation

The Random Forest model achieves strong performance on operationally well-defined queues. Service Outages and Maintenance (F1=0.684) and Technical Support (F1=0.608) are reliably classified because their vocabulary is domain-specific and relatively non-overlapping with other queues.

The three queues with the weakest RF F1 scores, Human Resources (0.395), Sales and Pre-Sales (0.422), and IT Support (0.459), share a common pattern: they are small in size (58–343 test examples each) and their vocabulary substantially overlaps with larger queues. For a support team, low recall on Human Resources (28%) means the majority of these tickets are silently misrouted to other teams, requiring agents to transfer them back. This is a wasted handoff that increases resolution time.

The contrast between the two models is instructive for model selection reasoning. On several minority classes, Logistic Regression outperforms Random Forest on F1: Human Resources (LR 0.441 vs RF 0.395), Service Outages and Maintenance (LR 0.705 vs RF 0.684), and Billing and Payments (LR 0.712 vs RF 0.696). These gains come from LR’s higher recall on small classes (General Inquiry: LR 0.45 vs RF 0.40, Human Resources: LR 0.45 vs RF 0.28). However, Random Forest’s stronger overall Macro F1 (0.527 vs 0.511) reflects better performance across the remaining classes. Macro F1 correctly penalises both failure modes equally, making it the right primary criterion for this selection decision.

5 Skills Design Rationale

5.1 File Structure and Mode Decisions

Each of the eight SKILL.md files follows a consistent structure: YAML frontmatter (name, mode, description), followed by six sections covering When to Use, How to Execute, Inputs from Agent State, Outputs to Agent State, Output Format, and Notes. The most consequential design decision in each file is the `mode` field. Full assignments and reasoning are shown in Appendix H.

5.2 Agent State Chain

The offline phase forms a chain: `preprocess-data` writes the fitted vectorizer, data splits, label encoder, and RAG corpus. `train-models` reads the splits and writes fitted models. `evaluate-models` reads models and test split and writes evaluation results. `select-model` writes `selected_model`, the fitted estimator that `classify-complaint` reads at inference time, bridging the offline and online phases.

In the online phase, `classify-complaint` reads `selected_model`, `tfidf_vectorizer`, and `label_encoder` from state and writes `predicted_queue`, `confidence_score`, and `ticket_type`. All three are declared as required inputs in `draft-response`, alongside `similar_tickets` and `order_context`. This explicit input/output contract means any developer can trace the full data lineage across all eight stages by reading the SKILL.md files alone.

5.3 Downstream Skill Design

The `draft-response SKILL.md` specifies that the LLM must combine all four inputs (predicted queue, confidence score, order context, and RAG matches) into a single business-facing output. This gives the LLM sufficient context to generate a personalised, grounded response.

The confidence threshold rule (below 50% triggers a manual review note) is specified in both `classify-complaint` and `draft-response SKILL.md` files. This ensures the rule is enforced at the inference stage and communicated in the downstream output, preventing the system from silently routing a low-confidence ticket without surfacing the uncertainty to the receiving agent.

6 System Evaluation

6.1 End-to-End Test Cases

Four test cases were selected to exercise distinct system behaviours: a clear high-confidence case, an ambiguous low-confidence case, an edge case blocked by the UI due to insufficient input, and a deliberate failure case where vocabulary mismatch causes misrouting. Full inputs and outputs are shown in Appendix I.

6.2 Failure Mode Analysis

Failure Mode 1: The Technical Cluster Confusion.

As shown in the appendix C, the most frequent systematic failure is misclassification within the three technical clusters: Product Support predicted as Technical Support (176 cases), Technical Support predicted as Customer Service (128 cases), and IT Support predicted as Technical Support (102 cases). Together these three confusion pairs account for over 400 misrouted tickets. A representative example: a ticket describing network connectivity problems was classified as Technical Support with 18.8% confidence. The true label was also Technical Support, but the low confidence reflects how closely the vocabulary overlaps with IT Support.

The root cause is vocabulary overlap. Product Support, IT Support, and Technical Support all discuss software failures, connectivity issues, system errors, and integration problems. The distinguishing signal (whether the issue is with the customer’s internal IT infrastructure, a specific product feature, or a general technical fault) is often absent from the ticket text itself. The business consequence is that tickets are routed between technically adjacent teams. The receiving agent must re-read the ticket, identify the mismatch, and initiate a transfer, adding unnecessary handling time.

Failure Mode 2: Semantically Hollow Tickets.

The second failure mode affects tickets using high-level business language with no domain-specific terms. An example: “Strategies for Brand Growth in Digital Space, seeking information on digital strategies to increase brand awareness.” This was classified as Customer Service (44.3% confidence). The true label was Product Support.

The root cause is vocabulary mismatch. Terms like “brand growth” and “digital strategies” rarely appear in Product Support training data, which skews toward technical language. When non-technical customers describe a product issue in general terms, they are routed to a general agent who may lack the product knowledge to resolve it, requiring an additional transfer.

7 Reflection and Improvement

7.1 What We Learned

Building RouteIQ showed us that the real challenge is not the model training itself, but designing the agent state contract and deciding when to use agentic AI versus deterministic logic. We realised that while LLMs offer great flexibility for ambiguous inputs, organisational logic is often better for simple lookups. In a commercial setting, balancing LLM latency and costs against business value would be the next logical step.

Model evaluation was more nuanced than expected. Random Forest’s high precision on minority classes (General Inquiry: 0.923) was misleading. The corresponding recall (0.300) revealed it rarely predicts those classes at all. Macro F1 correctly captured this: a model that almost never predicts a class cannot be considered competent on it, regardless of precision.

Finally, adding the RAG component (Node 6) significantly improved response quality by providing historical context. By reusing the TF-IDF vectoriser from Node 1, we kept the system lightweight and avoided extra dependencies while adding clear business value through more specific, grounded recommendations.

7.2 Proposed Improvements

Model: Replacing TF-IDF with a multilingual sentence embedding model as paraphrase-multilingual-MiniLM-L12-v2 would capture semantic similarity across paraphrases, directly addressing the vocabulary mismatch driving Failure Mode 2.

Pipeline: The confidence threshold of 50% is currently hardcoded. A more robust system would learn this threshold from validation data, selecting the cutoff that maximises a business-weighted objective. Additionally, replacing the mock CRM in Node 7 with a live API integration would make the customer personalisation in **draft-response** reflect real account data rather than synthetic records. Finally, adding an LLM-as-a-judge node at the end of the pipeline would enable fully automated reply dispatch for high-confidence, well-grounded cases, a natural next step if the system is extended beyond routing recommendations to end-to-end ticket resolution. A feedback loop could further improve the system over time. Once a ticket is resolved, the final resolution and any transfers could be fed back into the training data and RAG corpus to continuously refine routing accuracy.

8 AI Usage Declaration

Claude Code (Anthropic, Claude Sonnet 4.6) was used for coding assistance (debugging and boilerplate) and to check the clarity and grammar of text the group had already written in full. All model selection, metric interpretation, architectural design, and analytical judgements are the group’s own; no part of the technical report, SKILL.md files, or presentation script was AI-generated.

A Pipeline Architecture Diagram

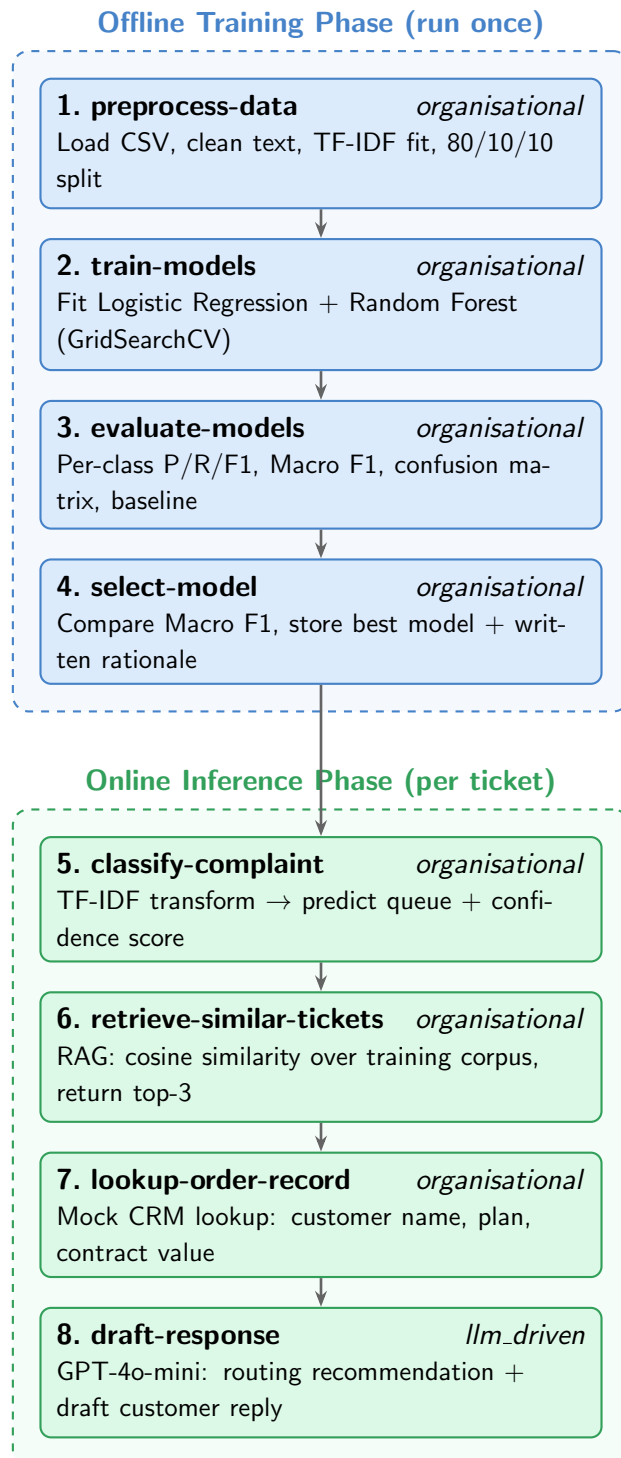


Figure 1: RouteIQ LangGraph pipeline: 8 agent nodes in execution order. Blue nodes are offline training stages; green nodes are online inference stage, process each incoming ticket at runtime, with node 8 as *llm-driven* node.

B Model Evaluation Tables

Table 1: Summary model comparison on held-out test set (2,858 tickets).

Model	Macro F1	Weighted F1	Accuracy
Logistic Regression	0.511	0.509	50.8%
Random Forest	0.527	0.537	53.8%
Baseline (majority)	0.045	0.132	29.3%

Table 2: Per-class Precision, Recall, and F1 on the held-out test set.

Queue	Logistic Regression			Random Forest		
	Prec	Rec	F1	Prec	Rec	F1
Billing and Payments	0.692	0.734	0.712	0.879	0.576	0.696
Customer Service	0.449	0.487	0.467	0.364	0.621	0.459
General Inquiry	0.529	0.450	0.486	0.842	0.400	0.542
Human Resources	0.433	0.448	0.441	0.696	0.276	0.395
IT Support	0.415	0.490	0.449	0.740	0.332	0.459
Product Support	0.438	0.408	0.422	0.512	0.410	0.455
Returns and Exchanges	0.410	0.521	0.459	0.800	0.417	0.548
Sales and Pre-Sales	0.391	0.489	0.435	0.493	0.370	0.422
Service Outages and Maintenance	0.636	0.791	0.705	0.690	0.678	0.684
Technical Support	0.602	0.483	0.536	0.540	0.695	0.608
Macro avg	0.500	0.530	0.511	0.656	0.478	0.527
Weighted avg	0.522	0.508	0.509	0.572	0.538	0.537

C Model Evaluation Figures

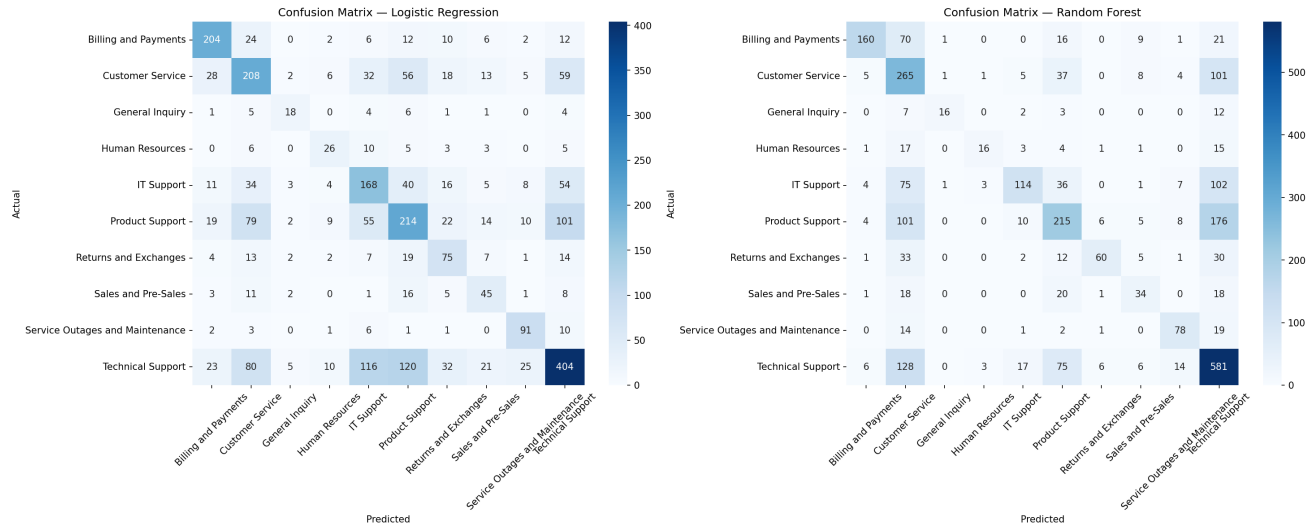


Figure 2: Side-by-side confusion matrix heatmaps for Logistic Regression (left) and Random Forest (right) on the held-out test set. Off-diagonal concentrations in the Technical Support / IT Support / Product Support cluster are visible in both models.

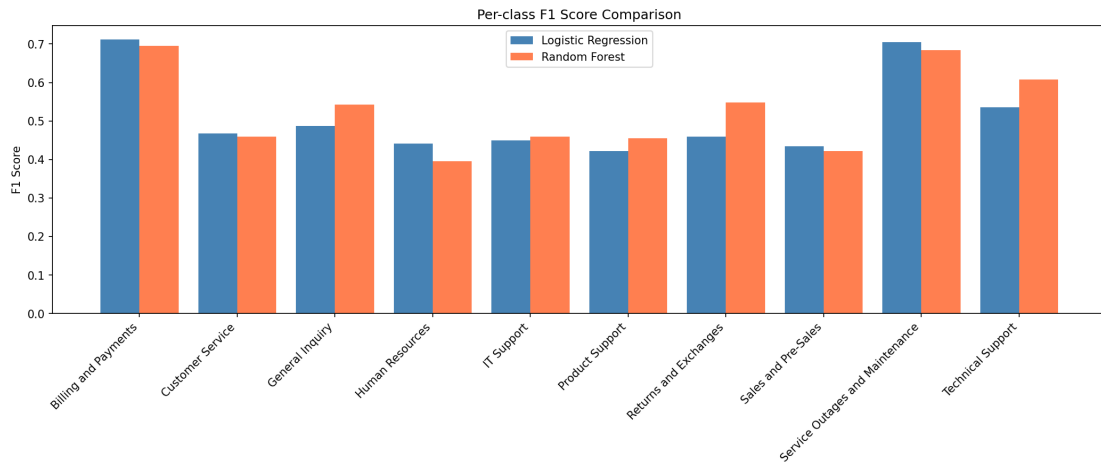


Figure 3: Per-class F1 scores for Logistic Regression and Random Forest. Random Forest achieves higher F1 on most queues; Logistic Regression's higher recall on small classes produces a more balanced but lower-precision result.

D Label Distribution

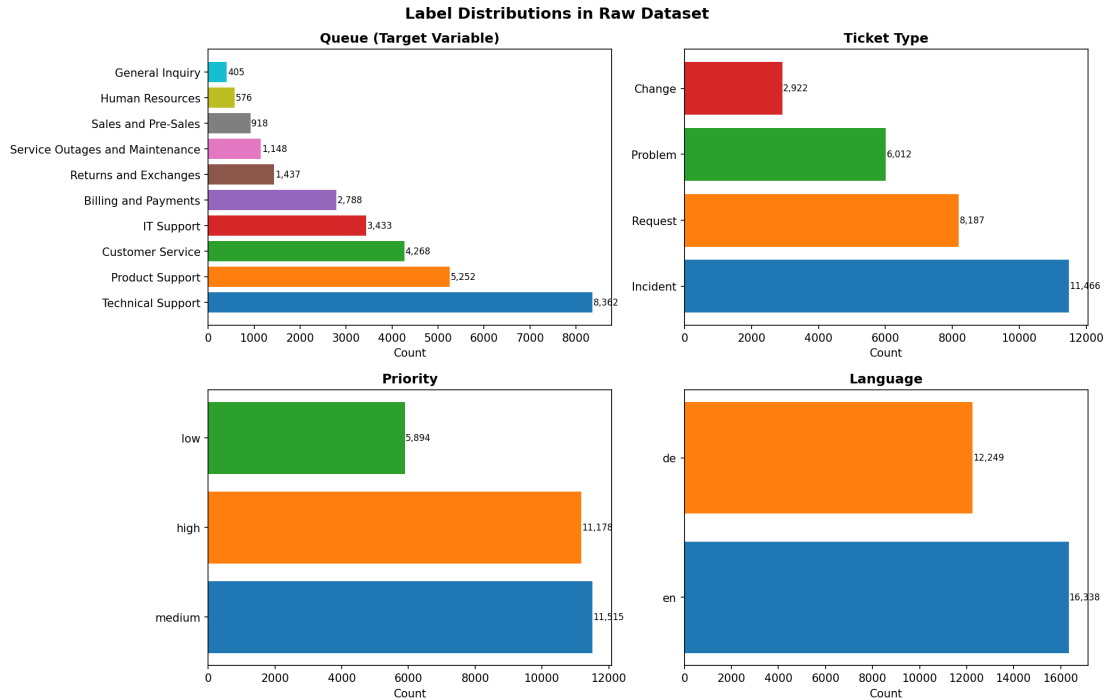


Figure 4: Queue label distribution across the full dataset (28,580 tickets). Technical Support is the majority class (8,362 tickets, 29.3%); General Inquiry is the smallest (405 tickets, 1.4%). Imbalance ratio: 20.6×

E Data Column Descriptions

Column	Type	Non-null	Description	Used in Model
subject	string	24,749	Ticket subject line written by the customer	Yes, concatenated with body
body	string	28,587	Full ticket body text	Yes, primary text input
answer	string	28,580	Agent replies to the ticket	No — excluded to prevent target leakage; used as RAG corpus
type	string	28,587	Ticket category (Incident, Request, Problem, Change)	No for ML; used by draft-response to shape reply tone
queue	string	28,587	Routing destination — which support team handles it	Yes, target variable
priority	string	28,587	Ticket urgency level (low/medium/high)	No for ML; available to draft-response for urgency framing
language	string	28,587	Language of ticket (en or de)	No for ML; handled via TF-IDF stopwords

Column	Type	Non-null	Description	Used in Model
version	int	28,587	Dataset version tag — constant value (51)	No, dropped — zero variance
tag_1	string	28,587	Multi-value topic tag	No — target leakage risk
tag_2	string	28,574	Additional topic tag	No — target leakage risk
tag_3	string	28,451	Additional topic tag	No — target leakage risk
tag_4	string	25,529	Additional topic tag (~11% missing)	No — target leakage risk, too sparse
tag_5	string	14,545	Additional topic tag (~49% missing)	No — target leakage risk, too sparse
tag_6	string	5,874	Additional topic tag (~79% missing)	No — target leakage risk, too sparse
tag_7	string	2,040	Additional topic tag (~93% missing)	No — target leakage risk, too sparse
tag_8	string	565	Additional topic tag (~98% missing)	No — target leakage risk, too sparse

F Model Configurations

Table 4: Model configurations and hyperparameter search.

Model	Key Settings	Search Space	Best Params
Logistic Regression	<code>class_weight='balanced'</code> <code>solver='lbfgs'</code> <code>max_iter=1000</code> interpretable and fast	$C \in \{0.01, 0.1, 1.0, 5.0, 10.0\}$	$C=10.0$
Random Forest	<code>class_weight='balanced'</code> <code>random_state=42</code> capture non-linearity	$n_estimators \in \{100, 200, 300\}$ $max_depth \in \{20, 40\}$	$n_estimators=300$ $max_depth=40$
Baseline	always predicts Technical Support	—	—

G TF-IDF Parameter Selection

Table 5: TF-IDF vectoriser configuration and rationale.

Parameter	Value	Rationale
<code>max_features</code>	10,000	Keeps vocabulary manageable; filters rare noisy terms
<code>ngram_range</code>	(1, 2)	Captures unigrams and bigrams for phrase-level signals (e.g. “billing statement”)
<code>sublinear_tf</code>	True	Log-scales term frequency to dampen the influence of high-frequency words
<code>min_df</code>	2	Ignores terms appearing in fewer than 2 documents, removing one-off noise
<code>analyzer</code>	word	Word-level tokenisation
<code>stop_words</code>	EN + DE	Removes function words from both English and German

Note: `strip_accents` is not set. German umlauts (ü, ä, ö) are preserved as distinct tokens, allowing the model to distinguish umlaut-bearing German terms from their non-umlaut counterparts.

H SKILL.md Mode Assignments

Table 6: SKILL.md mode assignments and design reasoning.

Skill	Mode	Reasoning
<code>preprocess-data</code>	organisational	Deterministic Python; fully reproducible given fixed data and seeds
<code>train-models</code>	organisational	GridSearchCV with fixed splits; no generative output
<code>evaluate-models</code>	organisational	Metric computation and plotting; no LLM involvement
<code>select-model</code>	organisational	Rule-based selection on Macro F1; written rationale is Python-generated
<code>classify-complaint</code>	organisational	Applies fitted sklearn model; declaring <code>llm_driven</code> would add API cost with no benefit
<code>retrieve-similar-tickets</code>	organisational	TF-IDF cosine similarity lookup; no generation required
<code>lookup-order-record</code>	organisational	DataFrame filter against mock CRM; purely deterministic
<code>draft-response</code>	llm_driven	Only stage where the LLM reads policy and generates a business-facing output

I End-to-End Test Cases

Table 7: End-to-end test case results on the RouteIQ pipeline.

Test Case	Input	Expected Business Output	True Label	Predicted	Outcome
High confidence	Discrepancies in Recent Billing Statement. Errors in the latest billing statement, hardware purchases incorrectly allocated across departmental accounts.	Route to Billing and Payments and generate personalised draft reply	Billing and Payments	Billing and Payments 73.0%	Correct. Unambiguous billing vocabulary. Personalised draft reply generated (Customer 60, Active, \$9,600 contract).
Ambiguous, low confidence	Network Connection Problems. Persistent connectivity issues disrupting workflow across several devices, possibly a malfunctioning repeater or router.	Route to Technical Support and flag for manual review	Technical Support	Technical Support 18.8% (<i>flagged</i>)	Correct route but low confidence. Vocabulary overlaps with IT Support. Manual review flag surfaced as intended.
Edge case: blocked by UI	Issue Encountered Assistance Needed	Reject input and request more detail	—	Blocked (too short)	Input rejected before reaching the pipeline. The UI enforces a minimum length check. System returns: “Please provide more details about your issue.”
Failure: vocabulary mismatch	Strategies for Brand Growth in Digital Space, seeking information on digital strategies to increase brand awareness.	Route to Product Support and provide product-specialist context	Product Support	Customer Service 44.3% (<i>flagged</i>)	Wrong queue. “Brand growth” and “digital strategies” skew toward Customer Service training data. Low confidence flag raised but routing is still incorrect.

References

- [1] Bueck, T. (2024). *Multilingual Customer Support Tickets*. Kaggle. <https://www.kaggle.com/datasets/tobiasbueck/multilingual-customer-support-tickets>
- [2] Lorikeet. (2026). *Cost per support ticket: Benchmarks and how to reduce it*. <https://www.lorikeetcx.ai/articles/customer-service-cost-per-ticket>
- [3] Unthread. (2026). *Support ticket tagging statistics*. <https://unthread.io/blog/support-ticket-tagging-statistics/>